



Educación
Secretaría de Educación Pública



TECNOLÓGICO
NACIONAL DE MÉXICO



Tecnológico Nacional de México Campus Felipe Carrillo Puerto

Ingeniería en Sistemas Computacionales



Asignatura

Introducción a la Graficacion

Tema

1.4 Evaluacion Practica

Profesor

MTI. Paloma Gongora Sabido

Alumno (s):

Tun Ek Javier Leonardo

(ISC-3B)

Felipe Carrillo Puerto a 13 de sept de 2026



Carretera Vigía Chico SN, C.P. 77200, Colonia Centro, Felipe Carrillo Puerto, Q. Roo
e-mail: direccion@itscarrillopuerto.edu.mx | tecnm.mx | carrillopuerto.tecnm.mx





Las herramientas de graficación permiten crear figuras, diseños, animaciones y representaciones visuales mediante código. En esta guía se presentan tres herramientas diferentes: **Processing**, **HTML/CSS** y **Matplotlib**.

1. **Processing**: dibujos, figuras geométricas y animaciones.
2. **HTML/CSS**: páginas web y diseño de elementos visuales.
3. **Matplotlib**: gráficas, datos y representaciones matemáticas.

1. Processing

Es un lenguaje y entorno de programación utilizado principalmente para crear **gráficos, figuras geométricas, animaciones y proyectos interactivos** mediante código.

2. HTML/CSS

HTML se utiliza para crear la estructura de las páginas web, mientras que **CSS** permite diseñar y modificar su apariencia, incluyendo colores, tamaños, posiciones y formas.

3. Matplotlib

Es una biblioteca de Python utilizada para crear gráficas, diagramas y representaciones visuales de datos y funciones matemáticas.





Processing

Iniciamos el proceso de graficación utilizando

```
VOID SETUP(){      void setup() {
Size ()            size(800, 600);
}
```

es una función de **Processing** que se utiliza solo una vez al iniciar el programa

Size funciona para definir el tamaño del lienzo

Luego seguimos con **void draw()**
Su función es dibujar constantemente

```
4
5 void draw() {
```

La función **background()**, sirve para cambiar el color de fondo del lienzo

La función **fill()** sirve para establecer el color del relleno utilizando el **RGB**

```
9
10 // SOL
11 fill(255, 255, 0);
12 noStroke();
13 ellipse(680, 80, 100, 100);
14
15 // NUBES
16 fill(255, 255, 255);
17 ellipse(150, 100, 80, 50);
18 ellipse(200, 80, 100, 70);
19 ellipse(250, 100, 80, 50);
20
```

Los parámetros son (x, y, ancho, alto) por lo tanto

150 es X
100 es Y
80 es ancho
50 es alto

noStroke() elimina los bordes de la figura

ellipse sirve para crear una figura en forma de elipse

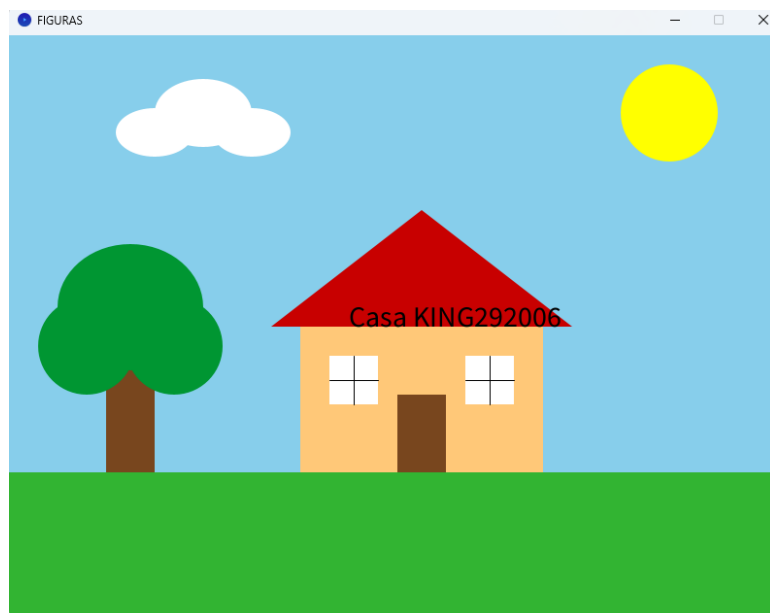
```

33
34 // CASA
35 fill(255, 200, 120);
36 rect(300, 300, 250, 150);
37
38 // TECHO
39 fill(200, 0, 0);
40 triangle(
41     270, 300,
42     425, 180,
43     580, 300
44 );
45
46 // PUERTA
47 fill(120, 70, 30);
48 rect(400, 370, 50, 80);
49
50 // VENTANA IZQUIERDA?
51 fill(255, 255, 255);
52 rect(330, 330, 50, 50);
53
54 // VENTANA DERECHA?
55 fill(255,255,255);
56 rect(470, 330, 50, 50);
57
58 stroke(0);
59 line(355, 330, 355, 380);
60 line(330, 355, 380, 355);
61
62 line(495, 330, 495, 380);
63 line(470, 355, 520, 355);
64
65 //texto
66 fill(0,0,0);
67 text("Casa KING292006" ,350,300);
68 textSize(30);
69
70 }

```

Rect sirve para crear una figura rectangular
Triangle sirve para crear una figura triangular
line dibuja una línea recta
Stroke sirve para definir el grosor de la línea
Text sirve para agregar texto dentro del lienzo
textSize sirve para establecer el tamaño del texto dentro del lienzo

En conclusión podemos decir que processing tiene las mismas funciones que p5.js





HTML/CSS

Iniciamos con la función **<!DOCTYPE html>** indica al navegador que el documento utiliza **HTML5**.

<html> es el element principal que contiene la pagina

<head> contiene la información y configuraciones de la pagina

body representa el contenido visible de la pagina

background-color sirve para cambiar el fondo del lienzo

```
<!DOCTYPE html>
<html>
<head>

    </div>

</html>
<style>
    body {
        background-color: rgb(51, 153, 255);
    }
</style>
</head>
<body>
```

1. **<div>** → crea un contenedor o elemento dentro de la página.
2. **style** → permite ponerle estilos CSS directamente.
3. **=** → asigna el contenido del atributo **style**.
4. **"** → indica dónde comienza y termina el código de estilos.
5. **Width**→ indica el ancho
6. **height**→indica el alto
7. **position** → sirve para colocar un tipo de posicionamiento
8. **left**→ sirve para colocar en la posición horizontal
9. **top**→ sirve para colocar en posición vertical
10. **PX**→ sirve para colocar el tamaño de píxeles

Estos comandos sirven para crear **un rectángulo**

```
<div style="
width:923px;
height:177px;
background-color:rgb(0, 153, 51);
position: absolute;
left: 0px;
top: 700px;
```



1. border-left: crea un triángulo de lado izquierdo y lo hace transparente
2. border-right: Crea el lado derecho y lo hace transparente
3. border-bottom: 100px solid rgb(102,255,51);Crea el triángulo de color verde

```
</div>

<div style="
  width: 0;
  height: 0;
  border-left: 50px solid transparent;
  border-right: 30px solid transparent;
  border-bottom:100px solid rgb(102, 255, 51);

  position: absolute;
  left: 60px;
  top: 400px;
">
```

border-radius: Hace que el cuadrado se convierta en **círculo**

```
<div style="
  width: 60px;
  height: 60px;
  background-color: rgb(255,255,255);
  border-radius: 50%;

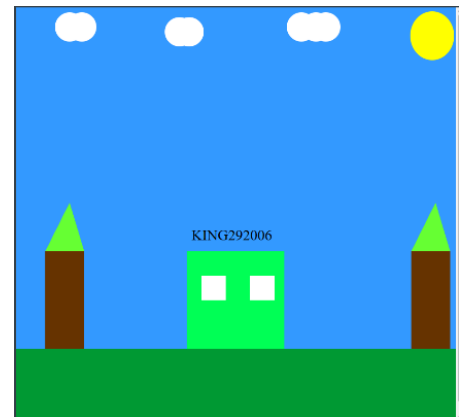
  position: absolute;
  left:605px;
  top: 10px;
"></div>
```

Este comando crea un Círculo

font-size: 30px; Hace que el texto tenga un tamaño de **30 píxeles**

KING292006: es el texto que aparece

```
<div style="
  position: absolute;
  left: 360px;
  top: 450px;
  font-size: 30px;
">
  KING292006
</div>
```





BLINDER-Python

Iniciamos utilizando **import matplotlib.pyplot as plt** esto Importa **Matplotlib**, específicamente **pyplot**, para poder crear y mostrar dibujos.

fig,ax =plt. Subplots()

sirve para crear un un lienzo con un sistema de ejes donde se va colocar las figuras

fig—representa el lienzo

ax—representa el área donde dibujamos

cabeza = plt.rectangle—crea un rectángulo

face color =("") sirve para rellenar la figura

ojo1 = plt.Rectangle() – esto crea un cuadrado

boca1 = plt.Rectangle()- esto crea un rectángulo

ax.set_xlim()—esto establece el limite horizontal

ax.set_ylim()---esto establece el limite vertical

ax.set_aspect("equal")—esto hace que la unidad en X tenga el mismo tamaño visual que Y

plt.title("CREEPERCITO")--- esto coloca el titulo encima del dibujo

plt.show()--- esto muestra el resultado final en el lienzo



```
import matplotlib.pyplot as plt

fig, ax = plt.subplots()

cabeza = plt.Rectangle((2, 2), 6, 5, facecolor="green")
ax.add_patch(cabeza)

ojo1 = plt.Rectangle((3, 5), 1.2, 1.2, facecolor="black")
ax.add_patch(ojo1)

ojo2 = plt.Rectangle((5.8, 5), 1.2, 1.2, facecolor="black")
ax.add_patch(ojo2)

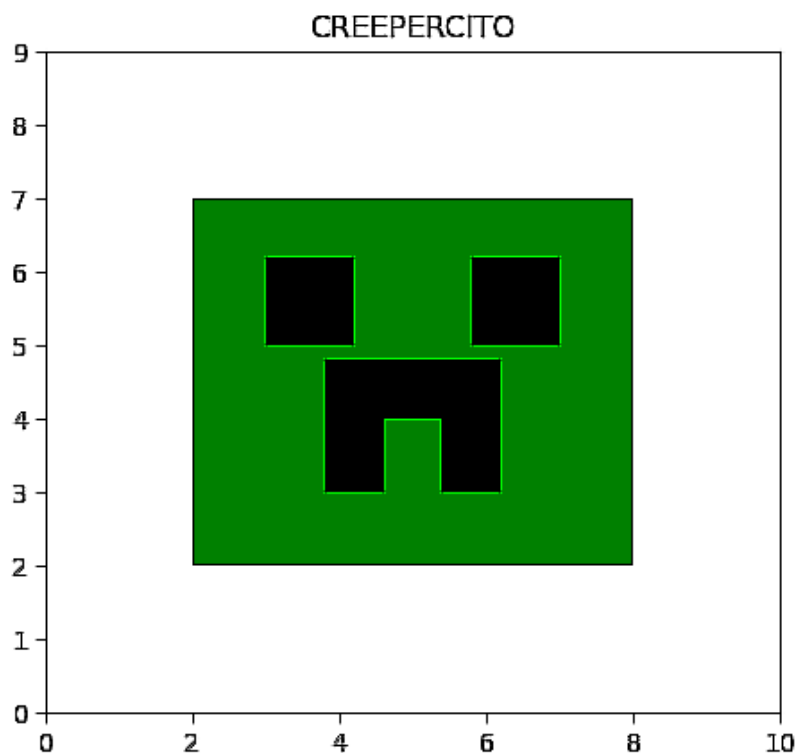
boca1 = plt.Rectangle((3.8, 4), 2.4, 0.8, facecolor="black")
boca2 = plt.Rectangle((3.8, 3), 0.8, 1.8, facecolor="black")
boca3 = plt.Rectangle((5.4, 3), 0.8, 1.8, facecolor="black")

ax.add_patch(boca1)
ax.add_patch(boca2)
ax.add_patch(boca3)

ax.set_xlim(0, 10)
ax.set_ylim(0, 9)

ax.set_aspect("equal")

plt.title("CREEPERCITO")
plt.show()
```





En conclusión processing es fácil de utilizar para principiantes ya que se parece al p5.js y se podría decir que tienen los mismos comandos para crear figuras , y tiene una fácil utilización al contrario de matplotlib que se necesita cierto conocimiento para poder usar esta extensión de python ya que utiliza comandos algo difíciles para utilizar para un principiante HTML se mantiene entre medio de ambas herramientas ya que no es tan difícil de poder utilizar pero requiere más conocimientos de códigos

Processing: está diseñado para la fácil creación de dibujos, figuras y animación

HTML/CSS: está más diseñada para crear páginas web y diseños

Matplotlib: está diseñada para crear gráficas, datos matemáticos

